

RVO（返回值优化） 和 **NRVO（命名返回值优化）** 是C++中的两种编译器优化技术，旨在减少不必要的对象拷贝，从而提高程序的性能。这两种优化主要涉及函数返回对象的方式。

1. 什么是RVO和NRVO？

- **RVO（Return Value Optimization，返回值优化）：**

RVO指的是编译器在函数返回临时对象时，直接在调用者的内存空间中构造该对象，避免了临时对象的创建和拷贝。这种优化适用于返回匿名临时对象的情况。

- **NRVO（Named Return Value Optimization，命名返回值优化）：**

NRVO是RVO的一种特例，适用于函数返回一个具名对象（即有名称的局部变量）。编译器通过在调用者的内存空间中直接构造这个具名对象，同样避免了不必要的拷贝。

2. 基于哪些C++标准？

- **C++11及之前的标准：**

在这些标准中，RVO和NRVO是可选的优化，编译器可以选择是否应用这些优化，但不保证一定会进行。

- **C++17及之后的标准：**

从C++17开始，标准明确规定在某些情况下必须进行**强制性拷贝省略**（也称为**强制性RVO**）。具体来说，当一个函数返回一个临时对象时，编译器必须直接在调用者的空间中构造该对象，彻底消除拷贝或移动操作。

3. 具体示例

下面通过几个示例来说明RVO和NRVO的应用。

示例1：RVO（返回临时对象）

```
#include <iostream>

class MyClass {
public:
    MyClass() { std::cout << "Constructor\n"; }
    MyClass(const MyClass&) { std::cout << "Copy Constructor\n"; }
    ~MyClass() { std::cout << "Destructor\n"; }
};

MyClass createObject() {
    return MyClass(); // 返回临时对象
}

int main() {
    MyClass obj = createObject();
    return 0;
}
```

输出（C++17及之后）：

```
Constructor
Destructor
```

解释：

在C++17及之后的标准中，createObject 函数返回一个临时 MyClass 对象。由于强制性RVO的存在，编译器会直接在 main 函数的 obj 内存空间中构造这个对象，完全省略了拷贝构造函数的调用。

示例2：NRVO（返回具名对象）

```
#include <iostream>

class MyClass {
public:
    MyClass() { std::cout << "Constructor\n"; }
    MyClass(const MyClass&) { std::cout << "Copy Constructor\n"; }
    ~MyClass() { std::cout << "Destructor\n"; }
};

MyClass createObject() {
    MyClass obj; // 具名对象
    return obj;   // 返回具名对象
}

int main() {
    MyClass obj = createObject();
    return 0;
}
```

输出（C++11及之前，可能有不同）：

```
Constructor
Copy Constructor
Destructor
Destructor
```

输出（C++17及之后）：

```
Constructor
Destructor
```

解释：

在C++17及之后的标准中，即使返回的是具名对象，编译器也可能应用NRVO或强制性拷贝省略，直接在 `main` 函数的 `obj` 内存空间中构造 `obj`，从而避免了拷贝构造函数的调用。

示例3：禁用拷贝和移动构造函数，强制使用RVO/NRVO

为了更清楚地看到RVO和NRVO的效果，可以禁用拷贝和移动构造函数：

```
#include <iostream>

class MyClass {
public:
    MyClass() { std::cout << "Constructor\n"; }
    MyClass(const MyClass&) = delete;           // 禁用拷贝构造
    MyClass(MyClass&&) = delete;                // 禁用移动构造
    ~MyClass() { std::cout << "Destructor\n"; }
};

MyClass createObject() {
    MyClass obj;
    return obj;
}

int main() {
    MyClass obj = createObject();
    return 0;
}
```

输出 (C++17及之后) :

```
Constructor
Destructor
```

解释:

由于拷贝和移动构造函数被删除，编译器必须应用NRVO或RVO来避免拷贝或移动操作，否则代码将无法编译。C++17及之后的标准确保在这种情况下，直接在 main 函数的 obj 内存空间中构造对象，从而成功编译。

总结

- **RVO** 和 **NRVO** 是C++编译器优化技术，用于减少对象拷贝，提升性能。
- **C++17** 标准引入了**强制性拷贝省略**，在更多情况下保证这些优化的应用。
- 通过合理编写代码，可以帮助编译器更好地应用RVO和NRVO，从而优化程序性能。